

Compte Rendu — Projet Lip Reading

De : Souhaieb El Amri À : Firas Jmili

Date : 1er Mai 2026

1. Résumé Exécutif

Ce document présente la stratégie finale adoptée pour le projet de détection automatique de la parole par lecture labiale (lip reading). Après analyse des échecs successifs de l’approche LipNet classique (CTC + CNN3D + BiGRU), nous retenons **deux approches complémentaires** :

- 1. **Approche A : MediaPipe + TCN sur MIRACL-VC1** — Solution légère, rapide, fonctionnelle immédiatement
- 2. **Approche B : AV-HuBERT pré-entraîné + fine-tuning sur petit dataset** — Solution SOTA, plus complexe, pour comparaison académique

2. Contexte et Problématique

2.1. Échecs de l’approche initiale (LipNet CTC)

L’architecture originale (Nick Nochnack / GRID corpus) présentait des limitations fondamentales :

Problème	Cause racine	Impact
Loss NaN	CTC avec labels paddés à 40 > 75 frames disponibles	Entraînement impossible
Même prédiction toujours	Normalisation [0,1] au lieu de z-norm par clip	Modèle aveugle au contenu
Crash GPU	Intel iGPU sélectionné au lieu de NVIDIA	Arrêt aléatoire
Mauvaise précision	2M paramètres, 20 mots, pas de contexte linguistique	40% max

2.2. Constat clé

Un modèle entraîné from scratch sur 20 phrases ne peut pas généraliser. La lecture labiale nécessite soit :

- Un pré-entraînement massif (4000+ heures)
- Des features robustes (landmarks géométriques plutôt que pixels)

3. Approche A : MediaPipe + TCN (Recommandée — Production)

3.1. Architecture

Vidéo (frames JPEG)

- MediaPipe Face Mesh (20 landmarks lèvres)
- Features géométriques (40D par frame : x,y normalisés)
- Resampling temporel (75 frames fixes)
- TCN (Temporal Convolutional Network)
- Classification softmax (20 classes MIRACL)

3.2. Pourquoi ça marche

Composant	Justification
MediaPipe Face Mesh	468 landmarks, robuste aux variations d'éclairage, pas besoin de recadrage manuel
Features géométriques	Invariant à la résolution, capture la forme des lèvres directement
TCN	Convolution dilatée voit tout le contexte temporel, entraîne 10× plus vite que LSTM
Classification	20 phrases fixes = problème fermé, pas besoin de CTC complexe

3.3. Implémentation

3.3.1. Fichier 1 : Extraction des landmarks

Fichier : 1_extract_miracl_landmarks.py

```
import cv2, numpy as np, mediapipe as mp, os, glob, pickle
from tqdm import tqdm

mp_face_mesh = mp.solutions.face_mesh.FaceMesh(
    static_image_mode=True, max_num_faces=1,
    refine_landmarks=True, min_detection_confidence=0.5
)

# 20 landmarks lèvres (MediaPipe)
LIP_INDICES = [61, 146, 91, 181, 84, 17, 314, 405, 321, 375,
               291, 308, 324, 318, 402, 317, 14, 87, 178, 88]

LABELS = {
    "words": {"01": "begin", "02": "choose", "03": "connection",
              "04": "navigation", "05": "next", "06": "previous",
              "07": "start", "08": "stop", "09": "hello", "10": "well done"},
    "phrases": {"01": "stop navigation", "02": "excuse me",
                 "03": "i am sorry", "04": "thank you", "05": "good bye",
                 "06": "i love this game", "07": "nice to meet you",
                 "08": "you are welcome", "09": "how are you",
                 "10": "have a good time"}
```

```

def extract_frame(image_path):
    img = cv2.imread(image_path)
    if img is None: return None
    h, w = img.shape[:2]
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    r = mp_face_mesh.process(rgb)
    if not r.multi_face_landmarks: return None

    lm = r.multi_face_landmarks[0].landmark
    return np.array([lm[idx].x for idx in LIP_INDICES] +
                    [lm[idx].y for idx in LIP_INDICES], dtype=np.float32)

def process_instance(folder, target=75):
    jpgs = sorted(glob.glob(os.path.join(folder, "color_*.jpg")))
    if not jpgs: return None

    feats = [f for f in [extract_frame(p) for p in jpgs] if f is not None]
    if len(feats) == 0: return None

    feats = np.array(feats)
    if len(feats) < target:
        pad = target - len(feats)
        feats = np.vstack([feats, np.repeat(feats[-1:], pad, axis=0)])
    else:
        idx = np.linspace(0, len(feats)-1, target, dtype=int)
        feats = feats[idx]

    return feats # (75, 40)

```

3.3.2. Fichier 2 : Modèle TCN

Fichier : 2_train_mediapipe_tcn.py

```

import tensorflow as tf
from tensorflow.keras import layers, models

def residual_block(x, dilation, filters):
    conv = layers.Conv1D(filters, 3, padding='causal',
                        dilation_rate=dilation)(x)
    conv = layers.BatchNormalization()(conv)
    conv = layers.ReLU()(conv)
    conv = layers.Conv1D(filters, 3, padding='causal',
                        dilation_rate=dilation)(conv)
    conv = layers.BatchNormalization()(conv)
    conv = layers.ReLU()(conv)

    if x.shape[-1] != filters:
        x = layers.Conv1D(filters, 1)(x)

    return layers.Add()([x, conv])

# Architecture
inputs = layers.Input(shape=(75, 40))
x = layers.Conv1D(64, 1)(inputs)

for i in range(4): # Dilatation 1, 2, 4, 8
    x = residual_block(x, 2**i, 64)

```

```

x = layers.GlobalAveragePooling1D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(20, activation='softmax')(x)

model = models.Model(inputs, outputs)
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

```

3.3.3. Fichier 3 : Inférence

Fichier : 3_predict.py

```

def predict_video(video_path):
    cap = cv2.VideoCapture(video_path)
    frames = []

    while True:
        ret, frame = cap.read()
        if not ret: break
        feat = extract_frame_from_video(frame) # Même MediaPipe
        if feat is not None:
            frames.append(feat)

    cap.release()
    if len(frames) == 0: return "NO FACE"

    # Resample + normaliser
    frames = resample(np.array(frames), 75)
    pred = model.predict(frames.reshape(1, 75, 40), verbose=0)
    return CLASSES[np.argmax(pred)]

```

3.4. Résultats Attendus

Métrique	Valeur	Notes
Précision test (MIRACL pur)	80-85%	20 phrases fixes, locuteurs variés
Précision test (votre visage)	90-95%	Après ajout de 50 enregistrements personnels
Temps d'inférence	< 50ms	CPU uniquement, pas besoin de GPU
Taille modèle	< 500 Ko	Embarquable sur mobile
Temps entraînement	2 min GPU / 10 min CPU	Rapide

4. Approche B : AV-HuBERT Pré-entraîné + Fine-tuning (Recherche)

4.1. Principe

Utiliser un modèle déjà entraîné sur 4 338 heures de vidéo (LRS3 + VoxCeleb2), puis ne ré-entraîner que la dernière couche sur un petit dataset.

Vidéo (MP4/AVI)

- Prétraitement (crop bouche 96×96, 75 frames)
- AV-HuBERT Backbone (gelé, 768 features)
- Classifieur linéaire (entraînable)
- 20 classes MIRACL (ou 500 mots LRW)

4.2. Pourquoi c'est différent

LipNet (Ancien)	AV-HuBERT (Nouveau)
Entraîné from scratch sur 20 phrases	Pré-entraîné sur 4 338 heures
CNN3D + BiGRU (2016)	Transformer + Self-Attention (2022)
2M paramètres	95M paramètres
CTC loss complexe	CTC avec beam search optimisé
Vocabulaire 20 mots	Vocabulaire 1 000+ mots
Précision 40%	Précision 70% zero-shot, 85% fine-tuné

4.3. Implémentation

4.3.1. Étape 1 : Téléchargement des poids

```
mkdir -p models && cd models
wget https://dl.fbaipublicfiles.com/avhubert/model/lrs3_vox/pretrained/base_lrs3_iter5.pt
wget https://dl.fbaipublicfiles.com/avhubert/data/dict.ltr.txt
cd ..
```

4.3.2. Étape 2 : Extraction de features (gelé)

```
from fairseq import checkpoint_utils
import torch

# Charger backbone pré-entraîné
models, cfg, task = checkpoint_utils.load_model_ensemble_and_task(
    ["models/base_lrs3_iter5.pt"],
    arg_overrides={"data": "."}
)
backbone = models[0].eval().cuda()

# Geler tous les paramètres
for param in backbone.parameters():
    param.requires_grad = False

# Extraction
```

```
with torch.no_grad():
    features = backbone.extract_fbank(video_tensor, None)
    # Sortie : (batch, time, 768)
```

4.3.3. Étape 3 : Classifier léger

```
class LightweightClassifier(torch.nn.Module):
    def __init__(self, input_dim=768, num_classes=20):
        super().__init__()
        self.pool = torch.nn.AdaptiveAvgPool1d(1)
        self.classifier = torch.nn.Sequential(
            torch.nn.Linear(input_dim, 256),
            torch.nn.ReLU(),
            torch.nn.Dropout(0.3),
            torch.nn.Linear(256, num_classes)
        )

    def forward(self, x):
        # x : (batch, time, 768)
        x = x.transpose(1, 2) # (batch, 768, time)
        x = self.pool(x).squeeze(-1) # (batch, 768)
        return self.classifier(x)
```

4.4. Dataset pour Fine-tuning

Option	Taille	Source	Avantage
MIRACL-VC1	3 000 instances	Votre archive	Gratuit, déjà là
LRW (Lip Reading Words)	500 mots, 1.5 GB	Oxford VGG	Vocabulaire plus large
Votre enregistrement	~50 vidéos	Webcam	Adapté à votre visage
GRID Corpus	33h, 33 locuteurs	Sheffield	Gratuit, phrases structurées

Recommandation : Combiner GRID (pré-entraînement initial) + vos enregistrements (fine-tuning final).

4.5. Résultats Attendus

Condition	Précision	Temps
Zero-shot (pas de fine-tuning)	60-70%	Immédiat
Fine-tune MIRACL (3 000 samples)	75-82%	2h GPU
Fine-tune GRID + votre visage	85-90%	4h GPU
Fine-tune LRS3 complet	92-95%	Impossible — dataset fermé

5. Comparaison et Choix

5.1. Matrice de Décision

Critère	Approche A : MediaPipe	Approche B : AV-HuBERT
Complexité setup	★ Basse (pip install)	★★★ Haute (fairseq, CUDA)
Temps développement	★ 1 jour	★★★ 3-4 jours
Précision MIRACL	★★★ 85%	★★ 80%
Généralisation (autres mots)	★ 0% (20 phrases fixes)	★★★ 70% (1000+ mots)
Inférence temps réel	★★★ Oui (<50ms)	★★ Non (200-500ms)
Embarqué (mobile/edge)	★★★ Oui (500 Ko)	★ Non (350 Mo)
Valeur académique	★★ Correct	★★★ Excellente (SOTA 2022)
Démonstration jury	★★ Fonctionnel	★★★ Impressionnant

5.2. Recommandation Stratégique

1. Pour la soutenance (démonstration live) : Approche A

- Fonctionne sur n'importe quel laptop
- Pas de dépendance GPU
- Précision suffisante pour impressionner

2. Pour le rapport écrit (composante recherche) : Approche B

- Montre que vous maîtrisez l'état de l'art
- Justifie les limitations de l'approche classique
- Ouvre vers des travaux futurs

6. Plan d'Action — Prochaines 48 Heures

6.1. Jour 1 (Aujourd'hui)

1. [] Exécuter `1_extract_miracl_landmarks.py` (10 min)
2. [] Entraîner `2_train_mediapipe_tcn.py` (5 min GPU / 15 min CPU)
3. [] Tester sur 3 vidéos de test (5 min)
4. [] Si précision < 75% : enregistrer vous-même 5 phrases, réentraîner

6.2. Jour 2 (Demain)

1. [] Télécharger poids AV-HuBERT (30 min)
2. [] Installer fairseq (1h, risque d'échec — prévoir alternative)
3. [] Extraire features MIRACL via AV-HuBERT (2h)

4. [] Entraîner classifieur linéaire (30 min)
5. [] Comparer précision A vs B dans tableau final

6.3. Jour 3 (Avant soutenance)

1. [] Préparer démo live : vidéo → prédiction en temps réel
2. [] Rédiger section “Comparaison des approches” dans le rapport
3. [] Sauvegarder tous les checkpoints et logs

7. Commandes Copier-Coller

7.1. Setup initial

```
# Créer environnement propre
conda create -n lipnet_final python=3.9 -y
conda activate lipnet_final

# Approche A dépendances
pip install mediapipe tensorflow opencv-python scikit-learn tqdm

# Approche B dépendances (si jour 2)
pip install fairseq==0.12.2 torch==1.13.1 torchvision torchaudio
```

7.2. Exécution Approche A

```
# 1. Extraction
python 1_extract_miracl_landmarks.py

# 2. Entraînement
python 2_train_mediapipe_tcn.py

# 3. Test
python 3_predict.py test_video.mp4
```

7.3. Exécution Approche B (Jour 2)

```
# Téléchargement poids
mkdir -p models && cd models
wget https://dl.fbaipublicfiles.com/avhubert/model/lrs3_vox/pretrained/base_lrs3_iter
5.pt
wget https://dl.fbaipublicfiles.com/avhubert/data/dict.ltr.txt
cd ..

# Extraction features
python extract_avhubert_features.py --input archive/dataset --output features/

# Fine-tune
python train_classifier.py --features features/ --epochs 20
```

8. Livrables Attendus

Livable	Description	Deadline
miracl_mediapipe_tcn.h5	Modèle A entraîné	Jour 1

Livrable	Description	Deadline
miracl_classes.npy	Mapping classes	Jour 1
avhubert_classifier.pt	Modèle B fine-tuné	Jour 2
demo_live.py	Démonstration temps réel	Jour 3
comparative_results.csv	Tableau précision A vs B	Jour 3
Rapport final	20 pages max, 2 approches	Jour 3

9. Risques et Mitigations

Risque	Probabilité	Mitigation
fairseq ne s'installe pas	Haute	Utiliser HuggingFace transformers comme fallback
Précision < 70% sur A	Moyenne	Ajouter données personnelles (50 enregistrements)
GPU indisponible jour 2	Moyenne	Approche A seule suffit pour soutenance
MIRACL mal prétraité	Basse	Vérifier 10 color_*.jpg par instance avant

10. Conclusion

L'approche MediaPipe + TCN (A) est la solution opérationnelle. L'approche AV-HuBERT (B) est la valeur ajoutée académique. Les deux ensemble forment un projet complet, démontrable, et techniquement solide.

Ne perdez plus de temps sur LipNet CTC. Exécutez l'Approche A aujourd'hui.
